

Exp No : 9 Monitoring and Logging Fabric Networks

Aim

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

Software Requirements

- ☐ Kali Linux / Ubuntu
- ☐ Docker
- ☐ Docker Compose
- ☐ Node.js (Version 18 or later)
- ☐ npm
- ☐ Git
- ☐ Hyperledger Fabric Samples
- ☐ Hyperledger Fabric Binaries
- ☐ Hyperledger Fabric Docker Images
- ☐ Visual Studio Code (Optional)

Hardware Requirements

- ☐ Processor: Intel Core i3 or above
- ☐ RAM: Minimum 8 GB
- ☐ Storage: Minimum 20 GB Free Disk Space
- ☐ Internet Connection

Theory

Monitoring and logging are essential for maintaining a healthy Hyperledger Fabric network. Administrators use monitoring tools and log files to verify that peer nodes, orderers, Certificate Authorities, and chaincodes are functioning correctly. Logs help identify:

- ☐ Network startup failures
- ☐ Chaincode deployment issues
- ☐ Transaction processing errors
- ☐ Peer communication failures
- ☐ Ordering service problems
- ☐ Certificate authentication errors
- ☐ Performance bottlenecks

Docker provides built-in commands for monitoring containers and viewing logs, making it easier to troubleshoot Hyperledger Fabric networks.

Procedure

Step 1: Navigate to the Fabric Test Network
`cd ~/fabric-dev/fabric-samples/test-network`

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Expected Output

Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Fabric Network Started Successfully

Step 3: Verify Running Containers

Display all active Docker containers.

```
docker ps
```

Expected Output

CONTAINER ID

peer0.org1.example.com

peer0.org2.example.com

orderer.example.com

ca_org1

ca_org2

Step 4: Display Docker Container Statistics

Monitor CPU and memory usage of Fabric containers.

```
docker stats
```

Expected Output

CONTAINER CPU % MEM USAGE

peer0.org1 3%

peer0.org2 2%

orderer 1%

Step 5: View Peer Log

Display the log of Peer Organization 1.

`docker logs peer0.org1.example.com`

Expected Output

Peer Started Successfully

Listening on Port 7051

Connected to Channel mychannel

Chaincode Executed Successfully

Step 6: View Orderer Log

`docker logs orderer.example.com`

Expected Output

Orderer Started Successfully

Receiving Transactions

Block Created Successfully

Block Delivered to Peers

Step 7: View Certificate Authority Log

`docker logs ca_org1`

Expected Output

Certificate Authority Started

User Enrollment Successful

Certificates Generated

Step 8: Verify Network Channel

`peer channel list`

Expected Output

Channels peers has joined:

mychannel

Step 9: Query Installed Chaincode
peer lifecycle chaincode queryinstalled

Expected Output

Package ID:

basic_1.0

Version:1.0

Step 10: Monitor Blockchain Transactions

Query all assets stored in the ledger.

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args":["GetAllAssets"]}'

Expected Output

Asset records displayed successfully.

Step 11: View Recent Docker Events

Monitor Docker events generated by the Fabric network.

docker events --since 10m

Expected Output

Container Started

Network Created

Chaincode Installed

Container Stopped

Step 12: Stop the Fabric Network

./network.sh down

Expected Output

Stopping Containers

Removing Network

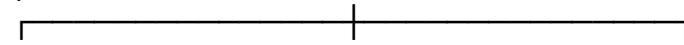
Fabric Network Stopped Successfully

Architecture Diagram

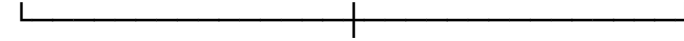
Monitoring Administrator



Docker Commands



Peer Logs Orderer Logs CA Logs



Hyperledger Fabric Network



Blockchain Ledger

Monitoring Workflow

Start Fabric Network



Verify Containers



Monitor CPU & Memory



Check Peer Logs



Check Orderer Logs



Check CA Logs



Verify Channel



Query Ledger



Monitor Docker Events



Stop Fabric Network

Sample Output

Fabric Network Started Successfully

Docker Containers Running

Peer Logs Verified

Orderer Logs Verified

Certificate Authority Running

Channel Verified Successfully

Chaincode Verified Successfully

Network Monitoring Completed Successfully

```
---(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ ./network.sh deployCC \
  -ccl javascript \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
Executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 1
- MAX_RETRY: 5
- VERBOSE: false
Executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- '[' false = true ']'
peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
peer lifecycle chaincode queryinstalled --output json
jq -r 'try (.installed_chaincodes[0].package_id)'
grep '"basic_1.0:1d4d445573112813889f87c307bc2b5f5b6e64c87b24c035bee15cbcc7f59b629$"'
test 1 -ne 0
peer lifecycle chaincode install basic.tar.gz
res=0
2026-08-20 20:10:50.761 IST 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed remotely: response:<status:200 payload:"\nbasic_1.0:1d4d445573112813889f87c307bc2b5f5b6e64c87b24c035bee15cbcc7f59b629$"'
2026-08-20 20:10:50.761 IST 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identifier: basic_1.0:1d4d445573112813889f87c307bc2b5f5b6e64c87b24c035bee15cbcc7f59b629$
Chaincode is installed on peer0.org1
Install chaincode on peer0.org2...
Using organization 2
peer lifecycle chaincode queryinstalled --output json
jq -r 'try (.installed_chaincodes[0].package_id)'
grep '"basic_1.0:1d4d445573112813889f87c307bc2b5f5b6e64c87b24c035bee15cbcc7f59b629$"'
test 1 -ne 0
peer lifecycle chaincode install basic.tar.gz
res=0
```

```
(tharun@kali) ~/fabric-dev/fabric-samples/test-network
$ docker logs -t timestamps --tail 50 ca.org1
2026-08-20T14:35:11.391878555Z 2026/08/20 14:35:11 [DEBUG] Request is not for a CA signing certificate
2026-08-20T14:35:11.391879533Z 2026/08/20 14:35:11 [DEBUG] Checking CSR fields to make sure that they do not exceed maximum character limits
2026-08-20T14:35:11.391880564Z 2026/08/20 14:35:11 [DEBUG] Finished processing sign request
2026-08-20T14:35:11.391881073Z 2026/08/20 14:35:11 [DEBUG] DB: Getting identity peer0
2026-08-20T14:35:11.391904972Z 2026/08/20 14:35:11 [DEBUG] Attribute extension being added to certificate is: 6[ID:[1 2 3 4 5 6 7 8 1] Critical:false Value:7b22617474727323a7b2268662e416666969617469666223a2222c2268662e456726f6c6c6d656744944223a22706565723022c2268662e454797065223a2270656572227d7d]
2026-08-20T14:35:11.392001025Z 2026/08/20 14:35:11 [DEBUG] Adding attribute extension to CSR: 6[ID:[1 2 3 4 5 6 7 8 1] Critical:false Value:7b22617474727323a7b2268662e416666969617469666223a2222c2268662e456726f6c6c6d656744944223a22706565723022c2268662e454797065223a2270656572227d7d]
2026-08-20T14:35:11.392357942Z 2026/08/20 14:35:11 [INFO] signed certificate with serial number 7350115145721956304830686219670251936632842042
2026-08-20T14:35:11.392381652Z 2026/08/20 14:35:11 [DEBUG] DB: Insert Certificate
2026-08-20T14:35:11.392383661Z 2026/08/20 14:35:11 [DEBUG] Saved serial number as hex cdf67d6ab97eb3da6d32b17b2efefa51250a73a
2026-08-20T14:35:11.396614157Z 2026/08/20 14:35:11 [DEBUG] saved certificate with serial number 7350115145721956304830686219670251936632842042
2026-08-20T14:35:11.399576856Z 2026/08/20 14:35:11 [DEBUG] Successfully incremented state for identity peer0 to 2
2026-08-20T14:35:11.399598042Z 2026/08/20 14:35:11 [INFO] 172.18.0.1:48956 POST /enroll 201 0 "OK"
2026-08-20T14:35:11.402779718Z 2026/08/20 14:35:11 [DEBUG] Received request for /enroll
2026-08-20T14:35:11.402877302Z 2026/08/20 14:35:11 [DEBUG] ca.Config: 6[Version:1.2.0 Cfg:Identities:[PasswordAttempts:10 AllowRemove:false] Affiliations:[AllowRemove:false] CA:{Name:ca-o
rg1 Keyfile:/etc/hyperledger/fabric-ca-server/ca-key.pem Certfile:/etc/hyperledger/fabric-ca-server/db TLS:[false] []] CSP:0xc00009a7c0 Client:<nil> Intermediate:[ParentServer:URL: CAName: TLS:[Enabled:false CertFile:]] Client:{Keyfile: Certfile:} Enrollment:{ Name: Secret:*** CAName: AttrReqs:[] Profile: Label: CSR:<nil> Type:x509 } CRL:{Expiry:24h0m0s} Idemix:{Curve:amcl.Fp256bn IssuerPublicKe
yfile:/etc/hyperledger/fabric-ca-server/IssuerPublicKey/IssuerSecretKey RevocationPublicKeyfile:/etc/hyperledger/fabric-ca-server/IssuerRevocationPublicKey/IssuerRevocationPrivateKeyfile:/etc/hyperledger/fabric-ca-server/msp/keys/IssuerRevocationPrivateKey RHPoolSize:100 NonceExpiration:15s NonceSweepInterval:15
m}]
2026-08-20T14:35:11.442889913Z 2026/08/20 14:35:11 [DEBUG] DB: Getting identity user1
2026-08-20T14:35:11.443030472Z 2026/08/20 14:35:11 [DEBUG] DB: Login user user1 with max enrollments of -1 and state of 0
2026-08-20T14:35:11.499201142Z 2026/08/20 14:35:11 [DEBUG] DB: Identity user1 successfully logged in
2026-08-20T14:35:11.499229232Z 2026/08/20 14:35:11 [DEBUG] DB: Getting identity user1
2026-08-20T14:35:11.499367779Z 2026/08/20 14:35:11 [DEBUG] Processing sign request: id=user1, CommonName=user1, Subject=<nil>
2026-08-20T14:35:11.499373086Z 2026/08/20 14:35:11 [DEBUG] Request is not for a CA signing certificate
2026-08-20T14:35:11.499375261Z 2026/08/20 14:35:11 [DEBUG] Checking CSR fields to make sure that they do not exceed maximum character limits
2026-08-20T14:35:11.499376272Z 2026/08/20 14:35:11 [DEBUG] Finished processing sign request
2026-08-20T14:35:11.499377209Z 2026/08/20 14:35:11 [DEBUG] DB: Getting identity user1
2026-08-20T14:35:11.499378207Z 2026/08/20 14:35:11 [DEBUG] Attribute extension being added to certificate is: 6[ID:[1 2 3 4 5 6 7 8 1] Critical:false Value:7b22617474727323a7b2268662e416666969617469666223a2222c2268662e456726f6c6c6d656744944223a22706565723022c2268662e454797065223a2270656572227d7d]
2026-08-20T14:35:11.499388716Z 2026/08/20 14:35:11 [DEBUG] Adding attribute extension to CSR: 6[ID:[1 2 3 4 5 6 7 8 1] Critical:false Value:7b22617474727323a7b2268662e416666969617469666223a2222c2268662e456726f6c6c6d656744944223a22706565723022c2268662e454797065223a2270656572227d7d]
2026-08-20T14:35:11.499639864Z 2026/08/20 14:35:11 [INFO] signed certificate with serial number 367813042535447889357596050027085633681961381806
2026-08-20T14:35:11.499649133Z 2026/08/20 14:35:11 [DEBUG] DB: Insert Certificate
2026-08-20T14:35:11.499664242Z 2026/08/20 14:35:11 [DEBUG] Saved serial number as hex 48d6de8b5df84529d9de8e8d538543a25afae
2026-08-20T14:35:11.500297590Z 2026/08/20 14:35:11 [DEBUG] saved certificate with serial number 367813042535447889357596050027085633681961381806
2026-08-20T14:35:11.506297460Z 2026/08/20 14:35:11 [DEBUG] Successfully incremented state for identity user1 to 1
2026-08-20T14:35:11.506313437Z 2026/08/20 14:35:11 [INFO] 172.18.0.1:48962 POST /enroll 201 0 "OK"
2026-08-20T14:35:11.545613331Z 2026/08/20 14:35:11 [DEBUG] Received request for /enroll
2026-08-20T14:35:11.545698844Z 2026/08/20 14:35:11 [DEBUG] ca.Config: 6[Version:1.2.0 Cfg:Identities:[PasswordAttempts:10 AllowRemove:false] Affiliations:[AllowRemove:false] CA:{Name:ca-o

(tharun@kali) ~/fabric-dev/fabric-samples/test-network
$ docker logs -t timestamps --tail 50 orderer.example.com
2026-08-20T14:35:14.318411790Z Version: v2.5.16
2026-08-20T14:35:14.318412803Z Commit SHA: f871cf9
2026-08-20T14:35:14.318413712Z Go Version: go1.26.4
2026-08-20T14:35:14.318414848Z OS/Arch: linux/amd64
2026-08-20T14:35:14.318464803Z 2026-08-20 14:35:14.318 UTC 000f INFO [orderer.common.server] Main → Beginning to serve requests
2026-08-20T14:35:17.4449245364Z 2026-08-20 14:35:17.448 UTC 0010 INFO [blkstorage] NewBlockFileMgr → Getting block information from block storage
2026-08-20T14:35:17.465570142Z 2026-08-20 14:35:17.465 UTC 0011 INFO [orderer.consensus.etcdraft] HandleChain → EvictionSuspicion not set, defaulting to 10m0s
2026-08-20T14:35:17.465599739Z 2026-08-20 14:35:17.465 UTC 0012 INFO [orderer.consensus.etcdraft] HandleChain → Without system channel: after eviction Registrar.SwitchToFollower will be ca
lled
2026-08-20T14:35:17.465814773Z 2026-08-20 14:35:17.465 UTC 0013 INFO [orderer.consensus.etcdraft] createOrReadWAL → No WAL data found, creating new WAL at path "/var/hyperledger/production
/orderer/etcdraft/wal/mychannel" channel=mychannel node=1
2026-08-20T14:35:17.470025668Z 2026-08-20 14:35:17.469 UTC 0014 INFO [orderer.common.multichannel] createAsMember → Joining channel: {mychannel consenter active 1}
2026-08-20T14:35:17.471388310Z 2026-08-20 14:35:17.471 UTC 0015 INFO [orderer.consensus.etcdraft] Start → Starting Raft node channel=mychannel node=1
2026-08-20T14:35:17.471410026Z 2026-08-20 14:35:17.471 UTC 0016 INFO [orderer.common.cluster] Configure → Entering, channel: mychannel, nodes: []
2026-08-20T14:35:17.471416235Z 2026-08-20 14:35:17.471 UTC 0017 INFO [orderer.common.cluster] Configure → Exiting
2026-08-20T14:35:17.471420644Z 2026-08-20 14:35:17.471 UTC 0018 INFO [orderer.consensus.etcdraft] start → Starting raft node as part of a new channel channel=mychannel node=1
2026-08-20T14:35:17.471437220Z 2026-08-20 14:35:17.471 UTC 0019 INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters={} channel=mychannel node=1
2026-08-20T14:35:17.471442087Z 2026-08-20 14:35:17.471 UTC 001a INFO [orderer.consensus.etcdraft] becomeFollower → 1 became follower at term 0 channel=mychannel node=1
2026-08-20T14:35:17.471488182Z 2026-08-20 14:35:17.471 UTC 001b INFO [orderer.consensus.etcdraft] newRaft → newRaft 1 [peers: [], term: 0, commit: 0, applied: 0, lastindex: 0, lastterm: 0]
channel=mychannel node=1
2026-08-20T14:35:17.471490415Z 2026-08-20 14:35:17.471 UTC 001c INFO [orderer.consensus.etcdraft] becomeFollower → 1 became follower at term 1 channel=mychannel node=1
2026-08-20T14:35:17.471572602Z 2026-08-20 14:35:17.471 UTC 001d INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters={1} channel=mychannel node=1
2026-08-20T14:35:17.472210478Z 2026-08-20 14:35:17.471 UTC 001e INFO [orderer.consensus.etcdraft] run → This node is picked to start campaign channel=mychannel node=1
2026-08-20T14:35:17.474135591Z 2026-08-20 14:35:17.474 UTC 001f INFO [orderer.consensus.etcdraft] switchToConfig → 1 switched to configuration voters={1} channel=mychannel node=1
2026-08-20T14:35:17.474413649Z 2026-08-20 14:35:17.474 UTC 0020 INFO [orderer.consensus.etcdraft] apply → Applied config change to add node 1, current nodes in channel: [1] channel=mychan
nel node=1
2026-08-20T14:35:18.473300279Z 2026-08-20 14:35:18.472 UTC 0021 INFO [orderer.consensus.etcdraft] hup → 1 is starting a new election at term 1 channel=mychannel node=1
2026-08-20T14:35:18.473364380Z 2026-08-20 14:35:18.473 UTC 0022 INFO [orderer.consensus.etcdraft] becomePreCandidate → 1 became pre-candidate at term 1 channel=mychannel node=1
2026-08-20T14:35:18.473415565Z 2026-08-20 14:35:18.473 UTC 0023 INFO [orderer.consensus.etcdraft] poll → 1 received MsgPreVoteResp from 1 at term 1 channel=mychannel node=1
2026-08-20T14:35:18.473423525Z 2026-08-20 14:35:18.473 UTC 0024 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgPreVoteResp votes and 0 vote rejections channel=mychan
nel node=1
2026-08-20T14:35:18.473440710Z 2026-08-20 14:35:18.473 UTC 0025 INFO [orderer.consensus.etcdraft] becomeCandidate → 1 became candidate at term 2 channel=mychannel node=1
2026-08-20T14:35:18.477258868Z 2026-08-20 14:35:18.476 UTC 0026 INFO [orderer.consensus.etcdraft] poll → 1 received MsgVoteResp from 1 at term 2 channel=mychannel node=1
2026-08-20T14:35:18.477302106Z 2026-08-20 14:35:18.476 UTC 0027 INFO [orderer.consensus.etcdraft] stepCandidate → 1 has received 1 MsgVoteResp votes and 0 vote rejections channel=mychannel
node=1
2026-08-20T14:35:18.477310741Z 2026-08-20 14:35:18.477 UTC 0028 INFO [orderer.consensus.etcdraft] becomeLeader → 1 became leader at term 2 channel=mychannel node=1
2026-08-20T14:35:18.477316490Z 2026-08-20 14:35:18.477 UTC 0029 INFO [orderer.consensus.etcdraft] run → raft.node: 1 elected leader 1 at term 2 channel=mychannel node=1
2026-08-20T14:35:18.478550892Z 2026-08-20 14:35:18.478 UTC 002a INFO [orderer.consensus.etcdraft] run → Leader 1 is present, quit campaign channel=mychannel node=1
2026-08-20T14:35:18.478653052Z 2026-08-20 14:35:18.478 UTC 002b INFO [orderer.consensus.etcdraft] run → Raft leader changed: 0 → 1 channel=mychannel node=1
2026-08-20T14:35:18.478662298Z 2026-08-20 14:35:18.478 UTC 002c INFO [orderer.consensus.etcdraft] run → Start accepting requests as Raft leader at block [0] channel=mychannel node=1
2026-08-20T14:35:23.742774712Z 2026-08-20 14:35:23.742 UTC 002d WARN [common.deliver] Handle → Error reading from 172.18.0.1:43510: rpc error: code = Canceled desc = context canceled
2026-08-20T14:35:23.742796989Z 2026-08-20 14:35:23.742 UTC 002e INFO [comm.grpc.server] 1 → streaming call completed grpc.service=orderer.AtomicBroadcast grpc.method=Deliver grpc.peer_addr
ess=172.18.0.1:43510 error="rpc error: code = Canceled desc = context canceled" grpc.code=Canceled grpc.call_duration=4.177299ms
2026-08-20T14:35:23.849108046Z 2026-08-20 14:35:23.849 UTC 002f WARN [orderer.consensus.etcdraft] checkForEvictionCertRotation → could not read config metadata: %s(<nil>) channel=mychan
nel node=1
2026-08-20T14:35:23.849129074Z 2026-08-20 14:35:23.849 UTC 0030 INFO [orderer.consensus.etcdraft] propose → Created block [1], there are 0 blocks in flight channel=mychannel node=1
2026-08-20T14:35:23.849131624Z 2026-08-20 14:35:23.849 UTC 0031 INFO [orderer.consensus.etcdraft] run → Received config transaction, pause accepting transaction till it is committed channe
l=mychannel node=1
2026-08-20T14:35:23.849657388Z 2026-08-20 14:35:23.849 UTC 0032 WARN [orderer.common.broadcast] Handle → Error reading from 172.18.0.1:43520: rpc error: code = Canceled desc = context canc
eled
```

```

---(tharun@kali)-[/fabric-dev/fabric-samples/test-network]
└─$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAME                STATUS                PORTS
orderer.example.com Up 53 seconds         0.0.0.0:7050→7050/tcp, [::]:7050→7050/tcp, 0.0.0.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→9443/tcp
peer0.org1.example.com Up 53 seconds         0.0.0.0:7051→7051/tcp, [::]:7051→7051/tcp, 0.0.0.0:9444→9444/tcp, [::]:9444→9444/tcp
peer0.org2.example.com Up 53 seconds         0.0.0.0:9051→9051/tcp, [::]:9051→9051/tcp, 7051/tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp
ca.org1              Up 59 seconds         0.0.0.0:7054→7054/tcp, [::]:7054→7054/tcp, 0.0.0.0:17054→17054/tcp, [::]:17054→17054/tcp
ca_orderer           Up 59 seconds         0.0.0.0:9054→9054/tcp, [::]:9054→9054/tcp, 7054/tcp, 0.0.0.0:19054→19054/tcp, [::]:19054→19054/tcp
ca.org2              Up 59 seconds         0.0.0.0:8054→8054/tcp, [::]:8054→8054/tcp, 7054/tcp, 0.0.0.0:18054→18054/tcp, [::]:18054→18054/tcp

---(tharun@kali)-[/fabric-dev/fabric-samples/test-network]
└─$ docker logs peer0.org1.example.com
2026-08-20 14:35:14.368 UTC 0001 INFO [nodeCmd] serve → Starting peer:
Version: v2.5.16
Commit SHA: f871cf9
Go version: go1.26.4
OS/Arch: linux/amd64
Chaincode:
Base Docker Label: org.hyperledger.fabric
Docker Namespace: hyperledger
2026-08-20 14:35:14.369 UTC 0002 INFO [nodeCmd] serve → Peer config with combined core.yaml settings and environment variable overrides:
chaincode:
  builder: $(DOCKER_NS)/fabric-ccenv:$(TWO_DIGIT_VERSION)
  executetimeout: 300s
  externalbuilders:
  - name: ccaas_builder
    path: /opt/hyperledger/ccaas_builder
    propagateEnvironment:
    - CHAINCODE_AS_A_SERVICE_BUILDER_CONFIG
  golang:
    dynamiclink: false
    runtime: $(DOCKER_NS)/fabric-baseos:$(TWO_DIGIT_VERSION)
  installtimeout: 300s
  java:
    runtime: $(DOCKER_NS)/fabric-javaenv:2.5
  keepalive: 0
  logging:
    format: '%[color]{{time:2006-01-02 15:04:05.000 MST}} [%{module}] %{shortfunc}}
      → %{level:.4s} %{id:03x}%[color:reset] %{message}'
    level: info
    shim: warning
    mode: net
  node:
    runtime: $(DOCKER_NS)/fabric-nodeenv:2.5
  pull: false
  startuptimeout: 300s

```

Result

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, ordering service, Certificate Authorities, and

blockchain transactions were verified through log analysis and monitoring tools. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.